

Práctica 2 - Estimación

Rodrigo Quirno Costa

2026-04-13

6. Colesterol

f) Estimadores EMV y empírico

```
colesterol <- c(
  1.53, 1.65, 1.72, 1.83, 1.62,
  1.75, 1.72, 1.68, 1.65, 1.61,
  1.70, 1.60, 1.73, 1.61, 1.52,
  1.81, 1.72, 1.50, 1.51, 1.65,
  1.58, 1.82, 1.65, 1.72, 1.65
)

n      <- length(colesterol)
mu_emv <- mean(colesterol)
sigma_emv <- sum((colesterol - mu_emv)^2) / n

cat("$\\hat{\\mu}_{EMV}$ =", round(mu_emv, 4), "\\n\\n",
    "$\\hat{\\sigma}_{EMV}$ =", round(sigma_emv, 4), "\\n\\n")
```

$$\hat{\mu}_{EMV} = 1.6612$$

$$\hat{\sigma}_{EMV} = 0.0083$$

```
xi      <- 1.73
p_emv   <- pnorm((xi - mu_emv) / sigma_emv)
p_mgm   <- sum(colesterol < xi) / n

cat("$P(X < 1.73)_{EMV}$ =", round(p_emv, 4), "\\n\\n",
    "$P(X < 1.73)_{MGM}$ =", round(p_mgm, 4), "\\n")
```

$$P(X < 1.73)_{EMV} = 1$$

$$P(X < 1.73)_{MGM} = 0.8$$

7. Estimadores de esperanza

b) EMV de theta y estimador de esperanza

```

s_a <- c(
  1.49, 1.12, 1.08, 0.65, 0.98, 0.86, 0.37, 0.41, 0.91, 0.85,
  0.25, 0.78, 0.30, 1.41, 0.18, 0.52, 0.40, 0.69, 0.73, 0.81
)

s_b <- c(
  1.32, 0.59, 1.41, 1.15, 0.34, 1.16, 1.29, 1.25, 1.18, 0.27,
  1.31, 0.96, 0.57, 0.66, 0.67, 1.34, 0.47, 1.07, 1.13, 0.25
)

# Calculamos el EMV de tita y estimador de esperanza basado en EMV
tita_hat_a <- sqrt(2 / mean(s_a))
esperanza_a <- mean(s_a)

cat("$\\hat{\\theta}_A$ =", round(tita_hat_a, 4), "\\n\\n",
    "$\\mathbb{E}[\\hat{\\theta}_A]$ =", round(esperanza_a, 4), "\\n\\n")

```

$$\hat{\theta}_A = 1.6445$$

$$\mathbb{E}[\hat{\theta}_A] = 0.7395$$

```

tita_hat_b <- max(s_b)
esperanza_b <- 2 * tita_hat_b / 3

cat("$\\hat{\\theta}_B$ =", round(tita_hat_b, 4), "\\n\\n",
    "$\\mathbb{E}[\\hat{\\theta}_B]$ =", round(esperanza_b, 4), "\\n\\n")

```

$$\hat{\theta}_B = 1.41$$

$$\mathbb{E}[\hat{\theta}_B] = 0.94$$

13. Distribución exponencial

a) ECDF vs CDF verdadera (n = 100)

```

set.seed(123)
n <- 100
lambda_real <- 1

data <- rexp(n, rate = lambda_real)

par(
  mar = c(3, 3, 2, 1),
  mgp = c(2, 0.7, 0)
)

plot(ecdf(data),
  main = paste("ECDF vs CDF verdadera - n =", n),
  xlab = "x",

```

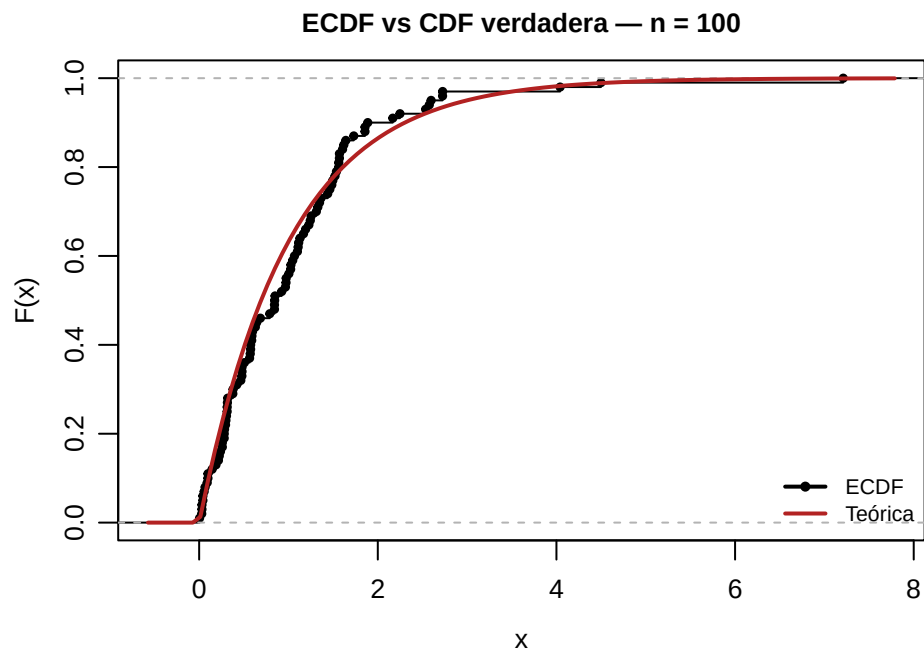
```

ylab      = "F(x)",
lwd       = 1,
cex       = 0.5,
cex.main  = 0.8,
cex.lab   = 0.8,
cex.axis  = 0.8
)

# CDF verdadera:  $F(x) = 1 - \exp(-\lambda * x)$ 
curve(pexp(x, rate = lambda_real),
      add = TRUE,
      col = "firebrick",
      lwd = 2
)

legend("bottomright",
      legend = c("ECDF", "Teórica"),
      col    = c("black", "firebrick"),
      lwd    = 2,
      lty    = c(1, 1),
      pch    = c(16, NA),
      cex    = 0.7,
      bty    = "n"
)

```



b) Estimador de λ (EMV = momentos)

```

lambda_hat <- 1 / mean(data)

cat("$\\hat{\\lambda}$ =", round(lambda_hat, 4), "\\n")

```

$$\hat{\lambda} = 0.9563$$

- c) Estimador alternativo de la FDA
 Propongo estimar usando KDE:

```
par(
  mar = c(3, 3, 2, 1),
  mgp = c(2, 0.7, 0)
)

plot(ecdf(data),
     main = paste("ECDF vs CDF verdadera - n =", n),
     xlab = "x",
     ylab = "F(x)",
     lwd = 1,
     cex = 0.5,
     cex.main = 0.8,
     cex.lab = 0.8,
     cex.axis = 0.8
)

curve(pexp(x, rate = lambda_real),
      add = TRUE,
      col = "firebrick",
      lwd = 2
)

# Usamos KDE (funcion density) e integramos
kde <- density(data, kernel = "g")

cdf_hat <- Vectorize(function(x) {
  h <- kde$bw
  mean(pnorm((x - data) / h) - pnorm(-data / h))
})

curve(cdf_hat(x),
      add = TRUE,
      col = "steelblue",
      lwd = 2
)

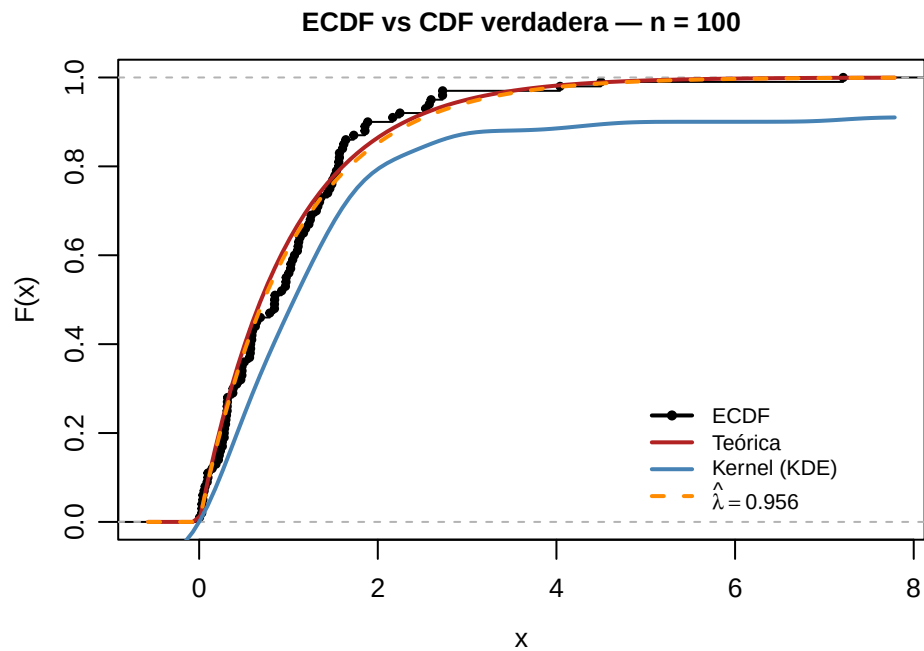
# FDA parametrica basada en el estimador de lambda
curve(pexp(x, rate = lambda_hat),
      add = TRUE,
      col = "darkorange",
      lwd = 2,
      lty = 2
)

legend(
  x = 5,
  y = 0.3,
  legend = as.expression(c(
    "ECDF",
```

```

    "Teórica",
    "Kernel (KDE)",
    bquote(hat(lambda) == .(round(lambda_hat, 3)))),
col   = c("black", "firebrick", "steelblue", "darkorange"),
lwd   = 2,
lty   = c(1, 1, 1, 2),
pch   = c(16, NA, NA, NA),
cex   = 0.7,
bty   = "n"
)

```



d) Convergencia para distintos tamaños muestrales

```

par(
  mfrow = c(1, 3),
  mar   = c(3, 3, 2, 1),
  mgp   = c(2, 0.7, 0)
)

for (n in c(10, 100, 1000)) {
  set.seed(123)
  data <- rexp(n, rate = lambda_real)

  plot(ecdf(data),
    main   = paste("n =", n),
    xlab   = "x",
    ylab   = "F(x)",
    lwd    = 1,
    col    = "black",
    cex    = 0.5,
    cex.main = 0.8,

```

```

    cex.lab = 0.8,
    cex.axis = 0.8
  )

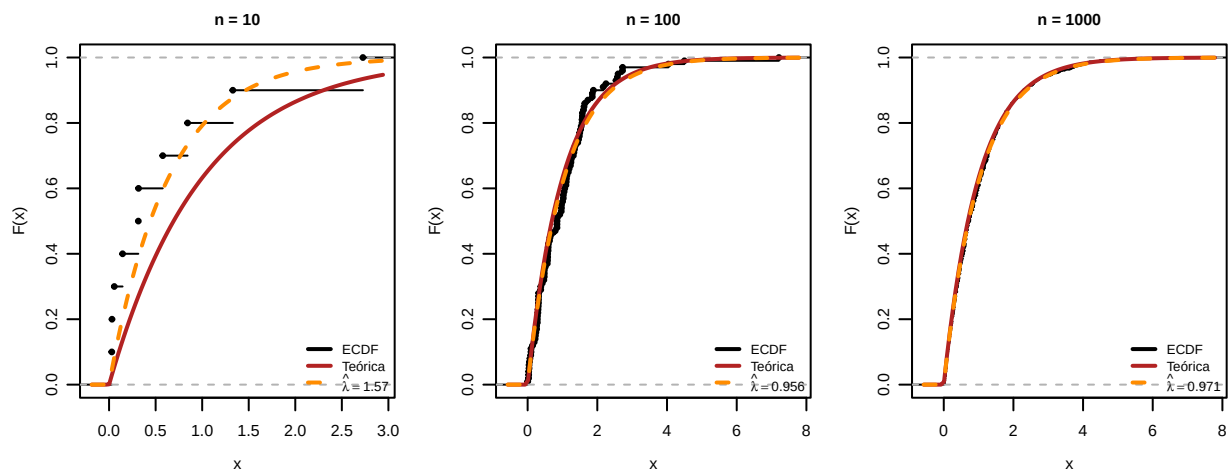
  curve(pexp(x, rate = lambda_real),
        add = TRUE,
        col = "firebrick",
        lwd = 2
  )

  lambda_hat <- 1 / mean(data)

  curve(pexp(x, rate = lambda_hat),
        add = TRUE,
        col = "darkorange",
        lwd = 2,
        lty = 2
  )

  legend("bottomright",
        legend = as.expression(c(
          "ECDF",
          "Teórica",
          bquote(hat(lambda) == .(round(lambda_hat, 3)))
        )),
        col = c("black", "firebrick", "darkorange"),
        lwd = 2,
        lty = c(1, 1, 2),
        cex = 0.6,
        bty = "n"
  )
}

```



```
par(mfrow = c(1, 1))
```

14. Comparación de estimadores de θ en $U(0, \theta)$

a) Estimadores de ECM

```
tita_0 <- 3
k      <- 1000
ns     <- c(6, 10, 20, 40, 80, 200)

# ECM empirico
ecm <- function(est, tita_0) mean((est - tita_0)^2)

resultados <- data.frame(
  n      = integer(),
  ECM_MV = numeric(),
  ECM_M  = numeric(),
  ECM_mod = numeric()
)

for (n in ns) {

  tita_mv <- numeric(k)
  tita_m  <- numeric(k)
  tita_mod <- numeric(k)

  for (i in 1:k) {
    muestra <- runif(n, min = 0, max = tita_0)

    tita_mv[i] <- max(muestra)
    tita_m[i]  <- 2 * mean(muestra)
    tita_mod[i] <- (n + 1) / n * max(muestra)
  }

  resultados <- rbind(resultados, data.frame(
    n      = n,
    ECM_MV = ecm(tita_mv, tita_0),
    ECM_M  = ecm(tita_m, tita_0),
    ECM_mod = ecm(tita_mod, tita_0)
  ))
}

kable(resultados,
  caption = "ECM empírico de los estimadores",
  col.names = c("n", "ECM MV", "ECM Momentos", "ECM Modificado"),
  align = c("c", "c", "c", "c"),
  digits = 6,
  row.names = FALSE
)
```

Table 1: ECM empírico de los estimadores

n	ECM MV	ECM Momentos	ECM Modificado
6	0.324967	0.483410	0.185291

n	ECM MV	ECM Momentos	ECM Modificado
10	0.137153	0.306910	0.071782
20	0.037976	0.147448	0.019433
40	0.009988	0.073389	0.005095
80	0.002736	0.036635	0.001421
200	0.000451	0.015215	0.000219

ECM teórico del estimador de momentos: $\hat{\theta}^m = \frac{\theta^2}{3n}$

```
tabla_teorico <- data.frame(
  n      = ns,
  ECM_M   = round(tita_0^2 / (3 * ns), 6),
  ECM_EMV = round(2 * tita_0^2 / ((ns + 1) * (ns + 2)), 6),
  ECM_mod = round(tita_0^2 / (ns^2 + 2 * ns), 6)
)

kable(tabla_teorico,
  caption = "ECM teórico de los estimadores",
  col.names = c("$n$", "ECM Momentos", "ECM MV", "ECM Modificado"),
  align     = c("c", "c", "c", "c"),
  row.names = FALSE,
  digits    = 6
)
```

Table 2: ECM teórico de los estimadores

n	ECM Momentos	ECM MV	ECM Modificado
6	0.5000	0.321429	0.187500
10	0.3000	0.136364	0.075000
20	0.1500	0.038961	0.020455
40	0.0750	0.010453	0.005357
80	0.0375	0.002710	0.001372
200	0.0150	0.000443	0.000223

b) i - iii. Boxplots estimadores

```
set.seed(42)
lista_mv <- list()
lista_m  <- list()
lista_mod <- list()

for (n in ns) {
  tita_mv <- numeric(k)
  tita_m  <- numeric(k)
  tita_mod <- numeric(k)
  for (i in 1:k) {
    muestra <- runif(n, min = 0, max = tita_0)
    tita_mv[i] <- max(muestra)
    tita_m[i] <- 2 * mean(muestra)
    tita_mod[i] <- (n + 1) / n * max(muestra)
  }
}
```



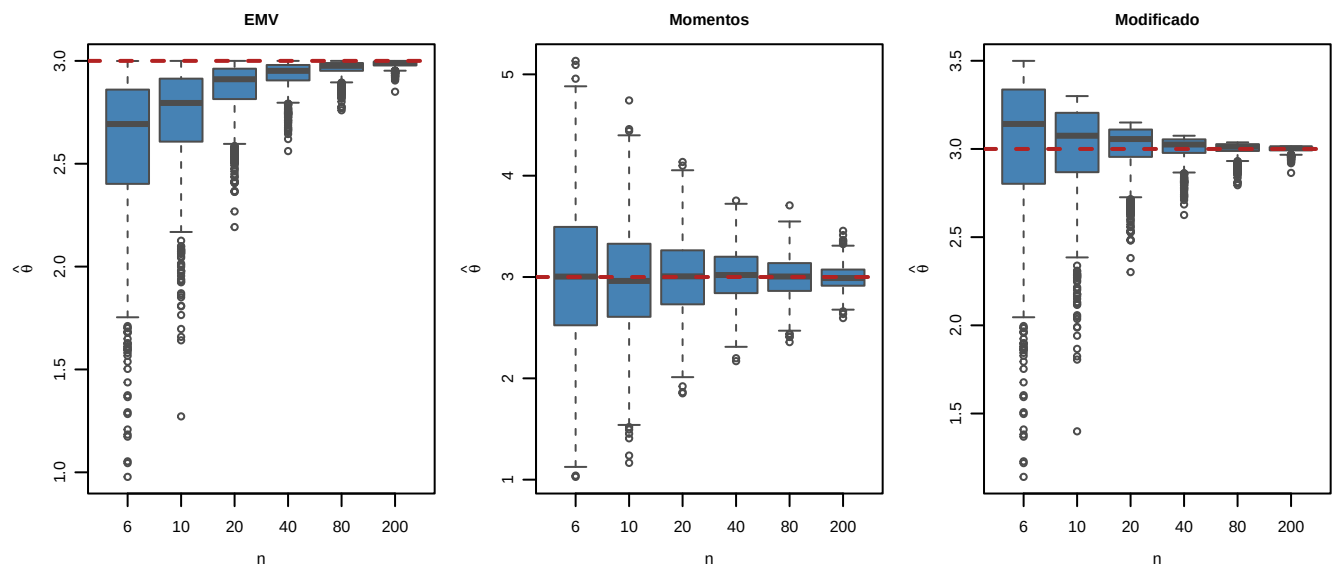
```

}
lista_mv[[as.character(n)]] <- tita_mv
lista_m[[as.character(n)]] <- tita_m
lista_mod[[as.character(n)]] <- tita_mod
}

par(mfrow = c(1, 3),
    mar = c(3, 3, 2, 1),
    mgp = c(2, 0.7, 0))

titulos <- c("EMV", "Momentos", "Modificado")
for (i in seq_along(list(lista_mv, lista_m, lista_mod))) {
  lst <- list(lista_mv, lista_m, lista_mod)[[i]]
  boxplot(lst,
    names = ns,
    main = titulos[i],
    col = "steelblue",
    border = "gray30",
    xlab = "n",
    ylab = expression(hat(theta)),
    cex.main = 0.8,
    cex.lab = 0.8,
    cex.axis = 0.8
  )
  abline(h = tita_0, col = "firebrick", lwd = 2, lty = 2)
}

```



iv. Curvas ECM estimado vs n

```

par(mar = c(3, 3, 2, 1),
    mgp = c(2, 0.7, 0)
)

```

```

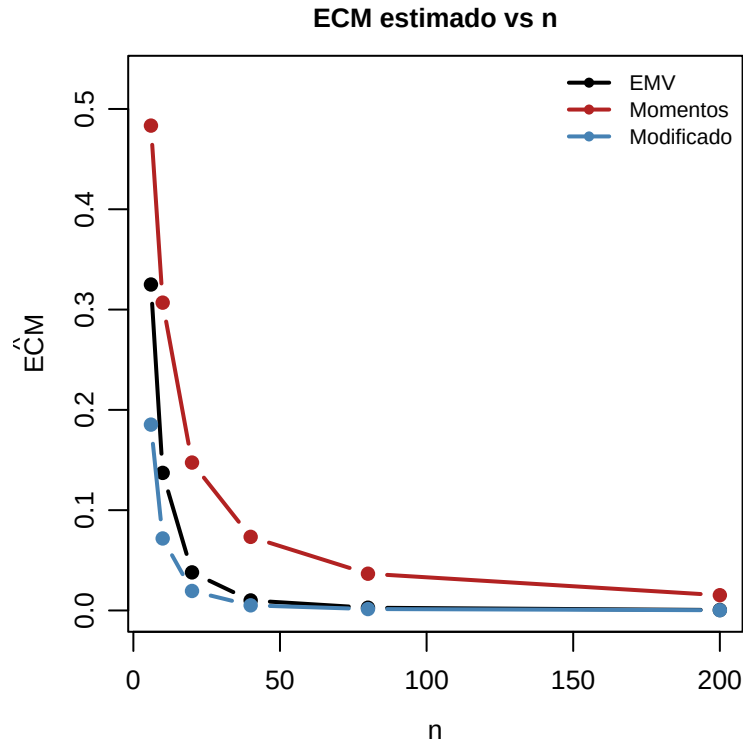
plot(ns, resultados$ECM_MV,
     type = "b",
     col = "black",
     lwd = 2,
     pch = 16,
     xlab = "n",
     ylab = expression(hat("ECM")),
     main = "ECM estimado vs n",
     ylim = c(0, max(resultados$ECM_M) * 1.1),
     cex.main = 0.8,
     cex.lab = 0.8,
     cex.axis = 0.8
)

lines(ns, resultados$ECM_M,
      type = "b",
      col = "firebrick",
      lwd = 2,
      pch = 16
)

lines(ns, resultados$ECM_mod,
      type = "b",
      col = "steelblue",
      lwd = 2,
      pch = 16
)

legend("topright",
      legend = c("EMV", "Momentos", "Modificado"),
      col = c("black", "firebrick", "steelblue"),
      lwd = 2,
      pch = 16,
      cex = 0.7,
      bty = "n"
)

```



15. Tormentas

- a) Hallar $\hat{\alpha}_M$, $\hat{\alpha}_{MV}$, $\hat{\lambda}_M$ y $\hat{\lambda}_{MV}$. Para el EMV, explorar el comando `fitdistr(x, densfun = "gamma")` del paquete MASS.

```
library(MASS)
lluvia <- scan("../Datos/tormenta.txt")
n <- length(lluvia)

alpha_m <- mean(lluvia)^2 / var(lluvia)
lambda_m <- mean(lluvia) / var(lluvia)

cat("$\\hat{\\alpha}_{M} =$", alpha_m, "\\n\\n",
    "$\\hat{\\lambda}_{M} =$", lambda_m, "\\n")
```

$$\hat{\alpha}_M = 1.972308$$

$$\hat{\lambda}_M = 0.1965625$$

```
ajuste_mv <- fitdistr(lluvia, densfun="gamma")

alpha_mv <- ajuste_mv$estimate["shape"]
lambda_mv <- ajuste_mv$estimate["rate"]

cat("$\\hat{\\alpha}_{MV} =$", alpha_mv, "\\n\\n",
    "$\\hat{\\lambda}_{MV} =$", lambda_mv)
```

$$\hat{\alpha}_{MV} = 1.961504$$

$$\hat{\lambda}_{MV} = 0.1954858$$

b) Estimación de $P(X > 20)$

```
p_mv <- pgamma(20, rate = lambda_mv , # plug-in
               shape = alpha_mv, lower.tail = FALSE) # TRUE => P(X <= q)

cat("$P_{\\hat{\\theta}_{MV}}(X > 20) =", p_mv, "\\quad",
    "\\hat{\\theta}_{MV}=(\\hat{\\alpha}_{MV},\\hat{\\lambda}_{MV})$")
```

$$P_{\hat{\theta}_{MV}}(X > 20) = 0.09398175, \quad \hat{\theta}_{MV} = (\hat{\alpha}_{MV}, \hat{\lambda}_{MV})$$

```
p_m <- pgamma(20, rate = lambda_m , shape = alpha_m, lower.tail = FALSE)

cat("$P_{\\hat{\\theta}_M}(X > 20) =", p_m)
```

$$P_{\hat{\theta}_M}(X > 20) = 0.09358515$$

```
p_emp <- mean(lluvia > 20)
cat("$\\hat{P}_{\\hat{\\theta}_n}(X > 20) =", p_emp)
```

$$\hat{P}_n(X > 20) = 0.115$$

c) Histograma + densidades

```
par(mar = c(3, 3, 2, 1),
    mgp = c(2, 0.7, 0)
)

hist(lluvia,
     freq      = FALSE,
     breaks    = 25,
     main      = expression(
       "Lluvia por tormenta - dist. " * Gamma(alpha, lambda)),
     xlab      = "Lluvia (mm)",
     ylab      = "Densidad",
     col       = "#D0E8F5",
     border    = "white",
     cex.main  = 0.8,
     cex.lab   = 0.8,
     cex.axis  = 0.8,
     xlim      = c(0, max(lluvia) * 1.05),
     las       = 1
)

kde <- density(lluvia, bw = "SJ") # SJ pq es más adaptativo
lines(kde, col = "#2C3E50", lw = 2)

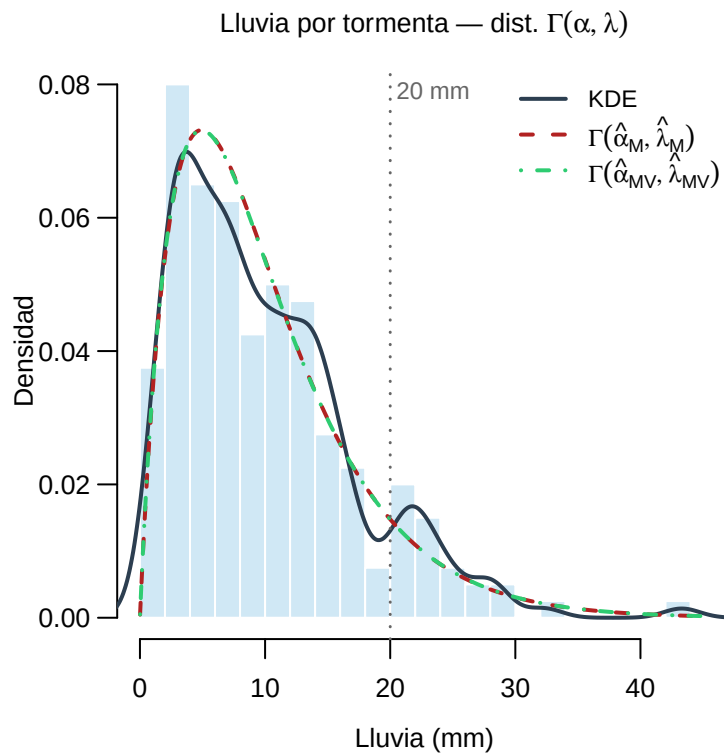
# secuencia para curvas paramétricas
xx <- seq(0.01, max(lluvia) * 1.05, length.out = 500)
```

```

lines(xx, dgamma(xx, rate = lambda_m , shape = alpha_m),
      col = "firebrick", lw = 2, lty = 2
)
lines(xx, dgamma(xx, rate = lambda_mv , shape = alpha_mv),
      col = "#2ECC71", lw = 2, lty = 4
)
# linea vertical
abline(v = 20, col = "grey40", lty = 3, lwd = 1.5)
text(20.5, par("usr")[4]*0.95, "20 mm",
     col = "grey40", cex = 0.75, adj = 0)

legend("topright",
      legend = c(
        "KDE",
        expression(Gamma(hat(alpha)[M], hat(lambda)[M])),
        expression(Gamma(hat(alpha)[MV], hat(lambda)[MV]))),
      col = c("#2C3E50", "firebrick", "#2ECC71"),
      lwd = 2,
      lty = c(1, 2, 4),
      bty = "n",
      cex = 0.78
)

```



Observaciones del gráfico:

- La KDE (negro) describe los datos sin asumir forma paramétrica y sigue fielmente el histograma.

- Las curvas Gamma MM (rojo) y Gamma MV (verde) son prácticamente indistinguibles, lo que es esperable para $n = 200$ ya que ambos estimadores son consistentes.
- El ajuste paramétrico es muy bueno, confirmando que la Gamma es un modelo adecuado para datos de lluvia.

18.c) Estimadores para $\mathcal{U}[0, \theta]$

i y ii.

```
k      <- 1000
tita   <- 1
epsilon <- 0.01
ns      <- c(10, 100, 500, 1000, 2000, 5000, 10000, 100000)

resultados <- data.frame(
  n      = integer(),
  P_mom  = numeric(),
  P_emv  = numeric()
)

for (n in ns) {

  pmom <- numeric(k)
  pemv  <- numeric(k)

  for (i in 1:k) {
    muestra <- runif(n, min = 0, max = tita)

    tita_mom <- 2 * mean(muestra)
    tita_emv <- max(muestra)

    pmom[i] <- abs(tita_mom - tita) < epsilon
    pemv[i] <- abs(tita_emv - tita) < epsilon
  }

  resultados <- rbind(resultados, data.frame(
    n      = n,
    P_mom  = mean(pmom),
    P_emv  = mean(pemv)
  ))
}

kable(resultados,
  caption = "Estimacion de  $P(|\hat{\theta}_n - \theta| < \epsilon)$ ",
  col.names = c("$n$", "$\hat{P}_{mom}$", "$\hat{P}_{emv}$"),
  align     = c("c", "c", "c"),
  digits    = 6,
  row.names = FALSE,
  format.args = list(scientific = FALSE)
)
```

Table 3: Estimacion de $P(|\hat{\theta}_n - \theta| < \epsilon)$

n	$\hat{P}_{mom}$	$\hat{P}_{emv}$
10	0.042	0.106
100	0.143	0.654
500	0.279	0.993
1000	0.426	1.000
2000	0.569	1.000
5000	0.767	1.000
10000	0.914	1.000
100000	1.000	1.000

20. CPU

```
cpu <- scan("../datos/cpu.txt")

n <- length(cpu)
sigma <- sd(cpu)

h_silverman <- 1.06 * sigma * n^(-1/5)

gaussian_kernel <- function(x, data, h) {
  u <- (x - data) / h
  1 / sqrt(2 * pi) * exp(-u^2 / 2)
}

kde <- function(x, data, h) {
  1 / h * mean(gaussian_kernel(x, data, h))
}

kde_default <- density(cpu)

grilla <- seq(min(cpu), max(cpu), length.out = 512) # 512 = 2^9
f_hat <- sapply(grilla, kde, data = cpu, h = h_silverman)

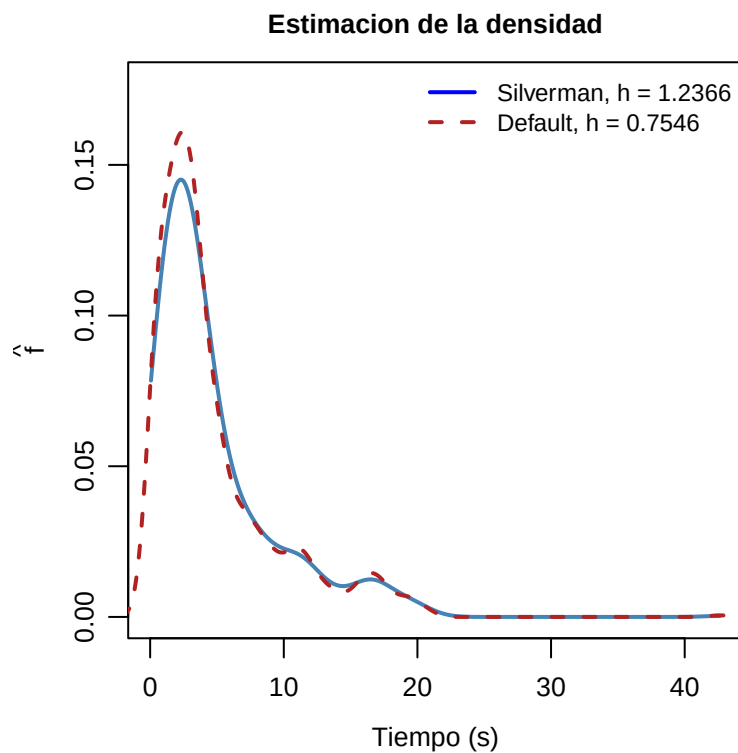
par(mar = c(3, 3, 2, 1),
     mgp = c(2, 0.7, 0))

plot(grilla, f_hat,
     main = "Estimacion de la densidad",
     type = "l",
     col = "steelblue",
     lwd = 2,
     xlab = "Tiempo (s)",
     ylab = expression(hat(f)),
     ylim = c(0, max(kde_default$y) * 1.1),
     cex.main = 0.8,
     cex.names = 0.8,
     cex.lab = 0.8,
     cex.axis = 0.8)
```

```
)

lines(kde_default, col = "firebrick", lwd = 2, lty = 2)

legend("topright",
      legend = c(paste("Silverman, h =", round(h_silverman, 4)),
                  paste("Default, h =", round(kde_default$bw, 4))),
      col     = c("blue", "firebrick"),
      lty     = c(1, 2),
      lwd     = 2,
      bty     = "n",
      cex     = 0.78
    )
)
```



21. LYVE1

```
datos <- read.csv("../datos/Debernardi.csv")

g1 <- datos$LYVE1[datos$diagnosis == 1]
g2 <- datos$LYVE1[datos$diagnosis == 2]
g3 <- datos$LYVE1[datos$diagnosis == 3]

h1 <- 1.06 * sd(g1) * length(g1)^(-1/5)
h2 <- 1.06 * sd(g2) * length(g2)^(-1/5)
h3 <- 1.06 * sd(g3) * length(g3)^(-1/5)
```



```

gaussian_kernel <- function(x, data, h) {
  u <- (x - data) / h
  1 / sqrt(2 * pi) * exp(-u^2 / 2)
}

kde <- function(x, data, h) {
  1 / h * mean(gaussian_kernel(x, data, h))
}

grilla <- seq(min(c(g1, g2, g3)), max(c(g1, g2, g3)), length.out = 512)

f1 <- sapply(grilla, kde, data = g1, h = h1)
f2 <- sapply(grilla, kde, data = g2, h = h2)
f3 <- sapply(grilla, kde, data = g3, h = h3)

par(mar = c(3, 3, 2, 1),
    mgp = c(2, 0.7, 0))

c1 <- "#669900"
c2 <- "#FFCC33"
c3 <- "firebrick"

plot(grilla, f1,
     main = "KDE Gaussiano",
     type = "l",
     col = c1,
     lwd = 2,
     xlab = "LYVE1",
     ylab = expression(hat(f)),
     ylim = c(0, max(f1, f2, f3) * 1.1),
     cex.main = 0.8,
     cex.names = 0.8,
     cex.lab = 0.8,
     cex.axis = 0.8
)

lines(grilla, f2, col = c2, lwd = 2)
lines(grilla, f3, col = c3, lwd = 2)

legend("topright",
     legend = c("diagnosis 1", "diagnosis 2", "diagnosis 3"),
     col = c(c1, c2, c3),
     lwd = 2,
     bty = "n",
     cex = 0.78
)

```

